SECURITY AUDIT OF

OP_NET Node

Public Report

Version: 1.1

March 19, 2026

Verichains Lab

info@verichains.io

https://www.verichains.io

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on **March 19, 2026**. We would like to thank the OP_NET Team for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

The audit focuses on identifying security vulnerabilities in the OP_NET Blockchain Node. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit process, the audit team had identified several vulnerable issues in the project.

TABLE OF CONTENTS

1. MANAGEMENT SUMMARY . . . 4
1.1. About OP_NET Node . . . 4
1.2. Audit Methodology . . . 4
1.3. Disclaimer . . . 5
1.4. Acceptance Minute . . . 5
2. AUDIT OVERVIEW . . . 6
2.1. Audit Scope . . . 6
2.2. Overview . . . 6
3. FINDINGS . . . 13
3.1. Flawed Validation Logic When Adding MLDSA Info to Cache Queue `[CRITICAL]` `[FIXED]` . . . 13
3.2. Decryption failure returns input message `[HIGH]` `[FIXED]` . . . 15
3.3. TaskId mismatch when handling message `[MEDIUM]` `[FIXED]` . . . 16
3.4. Incorrect worker recreation logic `[MEDIUM]` `[FIXED]` . . . 17
3.5. Incorrect logic in `getNextVMManager` `[MEDIUM]` `[FIXED]` . . . 19
3.6. Flawed Transaction Grouping Logic in Block Transaction Processing `[MEDIUM]` `[FIXED]` . . . 20
3.7. Gas Limit Check Uses Stale `gasUsed` Value `[MEDIUM]` `[FIXED]` . . . 22
3.8. Lookup Failure When Tweaked Public Key Is Used Instead of Original `[MEDIUM]` `[FIXED]` . . . 24
3.9. Missing `vmEvaluator` Cleanup Call `[LOW]` `[FIXED]` . . . 26
3.10. Incorrect Declaration of `subChannels` `[LOW]` `[FIXED]` . . . 27
3.11. Incorrect SSH Key Verification `[LOW]` `[FIXED]` . . . 28
3.12. `decrementPendingRequests()` may be called twice `[LOW]` `[FIXED]` . . . 29
3.13. Race Condition Can Prevent Cache Purge `[LOW]` `[FIXED]` . . . 30
3.14. Incorrect Memory Allocation for `winner.solutionHash` `[LOW]` `[FIXED]` . . . 31
3.15. Problems handling `attestations` in `generateTree()` `[LOW]` `[FIXED]` . . . 32
3.16. Inconsistent Logic Between `fromLevel` and `toLevel` Functions `[LOW]` `[FIXED]` . . . 33
3.17. Flawed Caching Logic in the `MLDSAPublicKeyCache` Class `[LOW]` `[FIXED]` . . . 34
3.18. Off-by-One Error in Remaining Byte Calculation `[LOW]` `[FIXED]` . . . 35
3.19. `exists()` length check breaks for hex string keys `[LOW]` `[FIXED]` . . . 36
3.20. MLDSA Link Request Not Stored When Block Has No Successful Transactions `[LOW]` `[FIXED]` . . . 37
3.21. Redundant Legacy Signature Verification in Link Request Process `[LOW]` `[ACKED]` . . . 39
3.22. Invalid Length Check for `targetHash` Parameter in `SubmitEpochRoute` `[LOW]` `[FIXED]` . . . 40
3.23. Missing `data.txid` produces empty buffer instead of 32-byte buffer `[LOW]` `[FIXED]` . . . 41
3.24. Inconsistent handling of the `address` and `addresses` parameters in the `GetLatestPendingTransactions` handler `[LOW]` `[FIXED]` . . . 42
3.25. Empty Array in `addresses` Parameter Suppresses Singular `address` Resolution `[LOW]` `[FIXED]` . . . 43
3.26. Use Loop Instead of Recursion for Retry Logic `[INFO]` `[FIXED]` . . . 44
3.27. Unsafe algorithms in SSH config `[INFO]` `[FIXED]` . . . 45
3.28. Vulnerabilities in Processing Transaction Input/Output Amounts Exceeding Limits `[INFO]` `[FIXED]` . . . 46
3.29. Error message incorrectly references SHA-1 instead of SHA-256 `[INFO]` `[FIXED]` . . . 47
4. VERSION HISTORY . . . 48

1. MANAGEMENT SUMMARY

1.1. About OP_NET Node

The **OP_NET Node** is a powerful protocol node designed to enable advanced smart contract functionalities on the Bitcoin blockchain. Its primary feature is providing a secure, decentralized execution environment where Bitcoin transactions can carry embedded smart contract logic. The node acts as the execution layer for these contracts, processing and confirming state transitions associated with Bitcoin transactions labeled specifically for this purpose. Through this, OP_NET extends the Bitcoin network's native capabilities by supporting programmable decentralized applications (dApps) while maintaining the highest security and finality guarantees of the Bitcoin mainnet.

OP_NET addresses this by acting as a true consensus layer rather than a meta-protocol, providing cryptographic proofs of correct execution so that all honest participants deterministically reach the same state from the same inputs. Contract bytecode, parameters, and execution data are all embedded into Bitcoin transactions, and Bitcoin miners still provide base-layer finality while OP_NET ensures that, given those on-chain inputs, only one resulting state is valid.

OP_NET nodes serve as critical infrastructure within the ecosystem, enabling interactions with smart contracts that resemble Ethereum's functionality but are tailored to Bitcoin's UTXO model. They allow for contract deployment, function calls, event logging, and persistent storage, all executed natively within Bitcoin transactions. This creates a bridge between Bitcoin's security and smart contract flexibility.

1.2. Audit Methodology

The security audit for the OP_NET node project is designed to systematically identify vulnerabilities, ensure protocol and implementation correctness, and validate secure operation of the node within the Bitcoin smart contract ecosystem. The audit methodology and focus areas are as follows:

Code Analysis
- Conduct a comprehensive manual and automated review of the source code, focusing on the Proof of Calculation (PoC) and Proof of Work (PoW) consensus mechanisms, virtual machine (VM) execution, and the REST API, JSON-RPC, and WebSocket layers.
- Verify conformance to development best practices and absence of security anti-patterns.

Static and Dynamic Testing
- Use static analysis tools to discover vulnerabilities such as memory safety issues, race conditions, and logic errors.

Functional and Security Testing
- Validate transaction validation, block synchronization, contract execution, and consensus operations.
- Assess API endpoints for authentication, authorization, input validation, and protection against injection or denial-of-service attacks.

Cryptographic and Incentive Review
- Audit cryptographic primitives and key management schemes critical to signing and consensus, including the integrity of post-quantum migration pathways (ML-DSA).
- Evaluate incentive structures and fee mechanisms to ensure fair and attack-resistant economic design.

Performance and Resilience Testing
- Benchmark under high load and simulate network partitions or reorganizations to verify robustness and fault-tolerance.

- Analyze memory retention patterns and cache/set cleanup cycles in long-running worker threads to prevent unbounded resource growth.

Documentation and Deployment Review

- Assess clarity and completeness of system documentation and deployment configurations to promote secure operations.

This structured audit approach targets security strengths and weaknesses in the OP_NET node's consensus, smart contract execution, and interfacing components to foster a resilient and secure deployment in the Bitcoin smart contract landscape.

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

Severity Level	Description
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

1.3. Disclaimer

OP_NET Team acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. OP_NET Team understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, OP_NET Team agrees that Verichains shall not be held responsible or liable, and shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.4. Acceptance Minute

This final report served by Verichains to the OP_NET Team will be considered an Acceptance Minute. Within **seven** days, if no any further responses or reports is received from the OP_NET Team, the final report will be considered fully accepted by the OP_NET Team without the signature.

2. AUDIT OVERVIEW

2.1. Audit Scope

This audit focused on identifying security flaws in code and the design of the OP_NET Node. It was first conducted on commit `24f9456d790f540dbcaa3ac9a1ddff4cbd70c167` from git repository `https://github.com/btc-vision/opnet-node`.

This table summarizes the report revision history, mapping each report version to its corresponding Git commit hash.

Report Version	Git Commit
0.1	`34b0ef5825baa1a65b9df90907807f08def66af4`
0.2	`99b96f68e080a03b651313035c9dc8df07299ed5`
0.3	`c86d30d2e4b54fa2933f01ac238ecd2ad6b5eb00`
0.4	`82c998c05a1de5d3604c435698e7ae10b7a64203`
0.5	`82c998c05a1de5d3604c435698e7ae10b7a64203`
0.6	`0e71eafbe0bf7f13540595c8a5d90cdc702b9840`
1.0	`ba067ff2581493f76b25cf770b9299cd7225f7f0`
1.1	`24f9456d790f540dbcaa3ac9a1ddff4cbd70c167`

2.2. Overview

The **OP_NET Node** is a sophisticated consensus and execution layer designed to support smart contract functionality directly on the Bitcoin blockchain. Built on **TypeScript** and **Node.js**, the protocol uses **libp2p** for resilient peer-to-peer networking and **Protocol Buffers (Protobuf)** for efficient, cross-process data serialization.

At its core, the system provides a deterministic execution environment where complex contract logic is embedded in native Bitcoin transactions. It is supported by a novel **Proof of Calculation (PoC)** mechanism—reinforced by a **Proof of Work (PoW)** fallback—to ensure strong network security and state finalization. Architecturally, the node uses a high-concurrency, multi-threaded engine: by isolating the indexer, mempool management, and virtual machine (VM) execution into discrete worker processes, it achieves high throughput while maintaining strict state consistency.

Beyond its execution capabilities, the OP_NET Node incorporates persistent contract storage and a pioneering post-quantum migration path through ML-DSA signatures. By mapping programmable logic to Bitcoin's UTXO model without compromising the mainnet's fundamental security guarantees, the OP_NET Node serves as the essential infrastructure for a new era of Bitcoin-native decentralized applications, bridging the gap between time-tested security and modern blockchain flexibility.

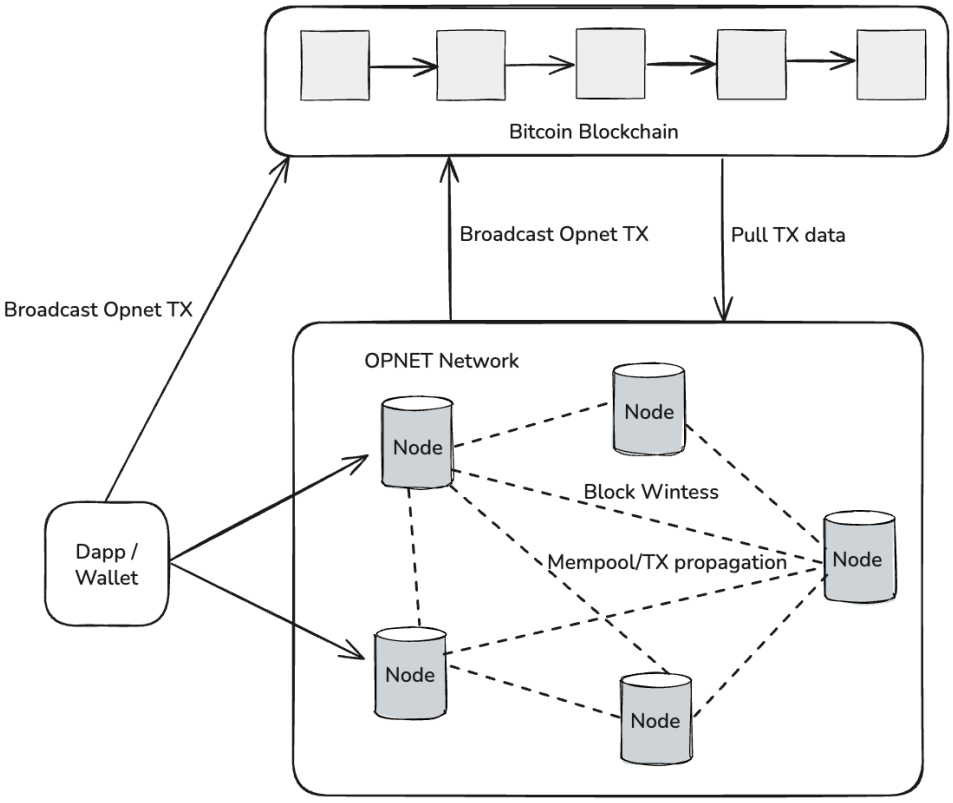

Figure 1: OP_NET Network Architecture

2.2.1. Core Features

The **OP_NET Node** implements a high-performance infrastructure designed for the scalable and secure execution of Bitcoin-native smart contracts. By operating as a deterministic consensus layer, the system enables complex decentralized applications to function directly on the Bitcoin blockchain, providing robust validator operations and trustless state management for the entire ecosystem.

- **Peer-to-Peer Network Interaction**: The node establishes persistent connections using a multi-layered libp2p architecture to enable efficient transaction and epoch propagation. It supports advanced peer discovery and custom communication framing to ensure reliable connectivity across the decentralized network.
- **Epoch and Transaction Management**: Handle proof-of-work epoch validation via EpochManager and RoswellConsensus, ensuring secure epoch progress. It processes transaction deserialization, validation, gas metering, execution, and state persistence while supporting nested contract calls and preventing reentrancy, ensuring robust transaction lifecycle management.
- **MLDSA Post-Quantum Key Linking**: To future-proof security, the system enables the link of legacy ECDSA keys to quantum-resistant MLDSA keys. This feature supports dual signature verification and provides flexible methods for transitioning user identities while maintaining on-chain privacy options.
- **Multi-Protocol API Suite**: The node provides a comprehensive interface for external interaction, including REST APIs for resource management, JSON-RPC for complex blockchain queries, and WebSocket services for real-time event streaming. These protocols integrate secure authentication and validation layers to ensure reliable dApp connectivity.

- **Persistent Storage with MongoDB**: Store blockchain ledger data persistently, including epochs, transactions, and state snapshots. Optimize database schema and indexes for efficient blockchain queries and retrieval.
- **State Management and Synchronization**: Maintain blockchain state and support mechanisms for state rollback or chain reorganization in case of forks or errors. Ensure consistent and reliable state updates across network nodes.
- **Data Sourcing from Bitcoin Network**: the node sources data directly from the Bitcoin blockchain, allowing each peer to independently recalculate and verify information at any given block, ensuring trustless and consistent data integrity.
- **Smart Contract Execution**: The OP_NET-VM provides a secure environment for executing complex smart contract logic. It ensures that all state changes are processed deterministically across the network, preventing divergent states and maintaining the overall integrity of the consensus layer.
- **Configuration and Operational Controls**: Support configurable environment variables, modular architecture for maintainability and scaling.

2.2.2. Consensus Mechanism

OP_NET implements a sophisticated **two-phase consensus model** that combines deterministic execution with proof-of-work finality, organized around Bitcoin's block structure. Time is divided into **epochs**, where each epoch spans exactly five consecutive Bitcoin blocks (approximately fifty minutes).

Phase 1: Proof of Calculation (PoC) — Deterministic State Building

Every OP_NET node independently monitors the Bitcoin blockchain and collects all confirmed transactions within a five-block epoch window. OP_NET enforces **canonical ordering** by sorting transactions deterministically: first by gas price, then by priority fees, then by transaction ID. This ensures all nodes process transactions identically.

Each node executes the ordered transactions through its local WebAssembly (WASM) virtual machine with fully deterministic execution—identical inputs always produce identical outputs. By the epoch's conclusion, every honest node independently arrives at an identical network state.

When an epoch ends, each node generates a **checksum root**—a cryptographic fingerprint of the entire state, including all balances, contract storage, and data. Even a single-bit difference produces a completely different checksum, making mismatches immediately detectable.

After miners submit proof-of-work solutions in the next epoch, every node deterministically selects the winner through a cascading mathematical calculation: most SHA1 collision bits, smallest public key, most key suffix bits, smallest salt, then smallest transaction ID. This guarantees all nodes select the same winner without communication.

Phase 2: Proof of Work (PoW) — Epoch Finality

After each epoch ends, miners compete to create an immutable checkpoint by finding SHA1 near-collisions. They hash the previous epoch's checksum with their public key and a 32-byte salt, rapidly changing the salt to maximize matching bits with the target. Winners submit their solutions as Bitcoin transactions with attestations to states 20+ blocks deep, preventing deep reorganization attacks. The winning miner becomes the official checkpoint but receives delayed rewards—earning gas fees from a future epoch rather than the current one. This delay incentivizes long-term network health and prevents epoch manipulation. Miners serve as witnesses, not validators.

2.2.3. Node Interaction in the OP_NET Network

Within the OP_NET ecosystem, nodes operate on a high-performance, decentralized peer-to-peer (P2P) network powered by the `libp2p` framework. At the architectural level, the protocol utilizes a multi-layered communication model where a custom ReusableStreamManager orchestrates a pool of persistent connections. By eliminating the overhead of repeated handshakes, this design ensures the low-latency propagation of Bitcoin blocks, mempool transactions, and contract execution outcomes across the network.

To guarantee data integrity, OP_NET enforces a custom binary framing protocol alongside a mandatory Acknowledgement (ACK) mechanism. This ensures that mission-critical payloads—such as block witness signatures and epoch validation proofs—are transmitted reliably.

Finally, OP_NET achieves network-wide state consistency through trustless synchronization. Each peer independently retrieves and verifies data from the Bitcoin mainnet to validate execution proofs. This approach helps the network reach deterministic consensus on smart contract state while fully inheriting the base-layer security and finality of the Bitcoin blockchain.

2.2.4. OP_NET's Post-Quantum Migration Solution

Quantum Threat

Quantum computers pose a profound threat to blockchain security by undermining the cryptographic primitives that current networks depend on. Their ability to rapidly solve complex mathematical problems could expose private keys from public keys, allowing the creation of forged digital signatures. Such capabilities would compromise transaction authentication, disrupt consensus mechanisms, and weaken the fundamental trust model of blockchain systems.

OP_NET's Solution

OP_NET addresses the quantum threat by implementing **ML-DSA (Module-Lattice-Based Digital Signature Algorithm)**, a NIST-standardized post-quantum cryptographic algorithm that provides quantum-resistant security. The solution uses a hybrid cryptographic system that integrates MLDSA with existing ECDSA infrastructure. This enables dual signature support—transactions can use ECDSA, MLDSA, or both simultaneously. The approach maintains full backward compatibility with existing ECDSA systems while enabling gradual migration to post-quantum cryptography, ensuring protection against future quantum attacks without disrupting current operations.

Key Linking Process

The solution uses a key linking mechanism that connects legacy ECDSA keys to MLDSA keys through on-chain transactions. Users create transactions with MLDSA link request features that include the hashed MLDSA public key (32-byte SHA-256), security level selection, and optionally the full MLDSA public key and signatures. Each transaction includes a legacy Schnorr signature proving ownership of the ECDSA key.

OP_NET's transaction processing infrastructure decodes these link request features and verifies signatures. For transactions without full public key exposure, the system verifies only the Schnorr signature on a message containing the linking data. For transactions with full public key exposure, the system performs dual verification—it checks both the Schnorr signature for ECDSA key ownership and the MLDSA signature for MLDSA key ownership and linkage.

After successful verification, OP_NET stores the linkage in the database with unique indexes on the hashed public key, legacy public key, and tweaked public key. The system records the block height for

historical tracking. This mechanism allows users to establish the relationship between their ECDSA and MLDSA keys while maintaining flexibility in privacy and verification needs.

Methods Supported by OP_NET

OP_NET provides two distinct methods for linking MLDSA keys to ECDSA keys, each designed to meet different privacy and operational requirements:

Method 1: Linking Without Exposing MLDSA Public Key

This method enables key linkage while maintaining privacy of the MLDSA public key. OP_NET processes link request transactions that contain the hashed MLDSA public key (32-byte SHA-256), security level selection, and a Schnorr signature proving ECDSA key ownership. The system verifies the Schnorr signature on a message containing the linking data and stores only the hash in the database. The full MLDSA public key remains private and can be exposed in a subsequent transaction if signature verification becomes necessary.

Method 2: Linking With Exposing MLDSA Public Key

This method establishes the key linkage while simultaneously exposing the full MLDSA public key on-chain. OP_NET processes link request transactions containing the hashed public key, full MLDSA public key, MLDSA signature, and Schnorr signature. The system performs dual signature verification: it verifies the Schnorr signature for ECDSA key ownership and the MLDSA signature for MLDSA key ownership and linkage establishment. Upon successful verification, OP_NET stores the linkage with the full public key exposed, enabling immediate use for operations requiring signature verification, such as epoch submissions.

Both methods are production-ready and operational, allowing users to choose the appropriate approach based on their privacy preferences and immediate verification requirements.

2.2.5. Node Components

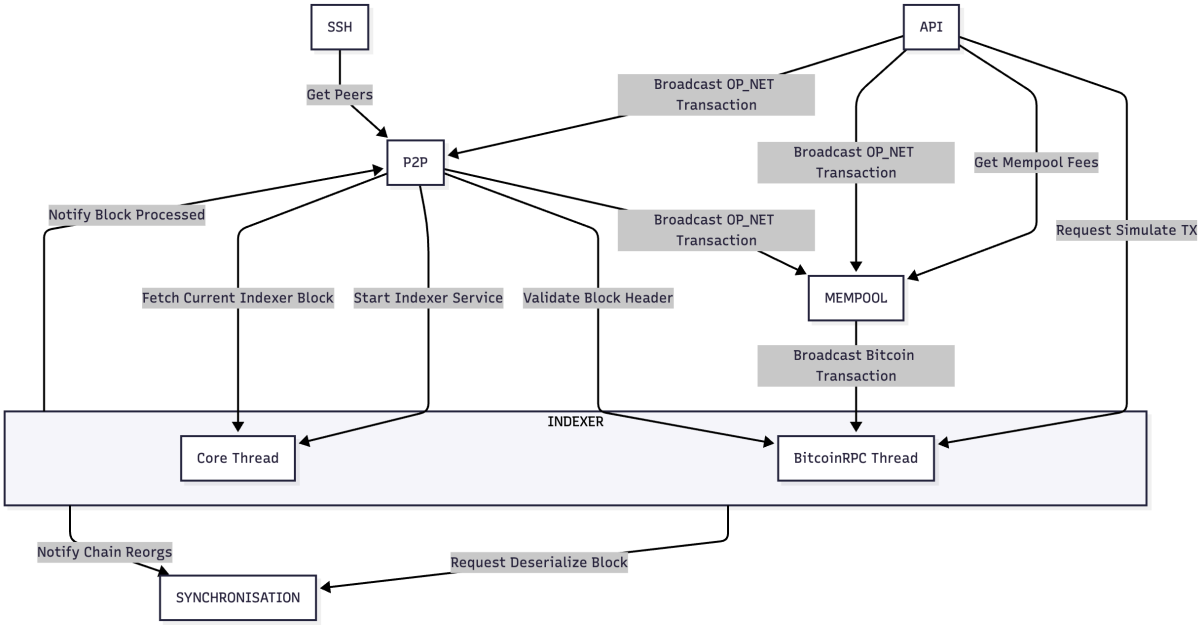

Figure 2: Component Interactions

These components work together through asynchronous message passing, coordinating block processing, transaction validation, contract execution, consensus validation, and data storage to create a reliable and efficient OP_NET blockchain node.

Consensus and Block Processing

- **OP_NET Consensus (Roswell)**: The primary logic engine that enforces protocol rules and manages Proof-of-Calculation (PoC) epochs, ensuring all participants converge on the same blockchain state.
- **BlockIndexer & ChainSynchronisation**: These modules facilitate the ingestion of Bitcoin blocks. They extract UTXOs, deserialize OPNET-specific transaction data, and maintain synchronization between the node and the Bitcoin mainnet.
- **EpochManager & BlockWitnessManager**: Coordinate the validation of epoch submissions and the collection of witness signatures. They ensure that block proofs are propagated and confirmed across the P2P network.

Execution and Virtual Machine

- **VMManager** & **ContractEvaluator**: Oversee the secure execution of smart contract bytecode. They manage environment setup, gas metering, and reentrancy protection to ensure deterministic state transitions.
- **RustContract** (WASM Bridge): Provides a sandboxed WebAssembly runtime implemented in Rust. This bridge allows the node to execute high-performance contract logic with strict memory safety and isolation from the main Node.js process.

Core Orchestration

- **JSONRpc2Manager** & **RPC Sub-workers**: These components handle external JSON-RPC requests by dispatching complex execution tasks to available sub-worker threads, enabling concurrent handling of multiple dApp interactions.

Storage and Mempool

- **VMMongoStorage** & **Repository Layer**: Provides a high-performance persistence layer using MongoDB. It stores the blockchain ledger, contract states, and transactional history with optimized indexing for rapid retrieval.
- **MempoolManager**: Monitors the Bitcoin mempool and manages OPNET's local pending transaction pool, performing pre-validation before broadcasting new transactions to peers.

2.2.6. Transaction Processing Flow

The OP_NET node processes transactions embedded within Bitcoin blocks through a series of coordinated steps involving multiple components:

1. Block Reception and Deserialization

The BlockIndexer detects new Bitcoin blocks via the Network Layer and initiates processing by sending a DESERIALIZE_BLOCK message to ChainSynchronisation. This component extracts OP_NET transactions and UTXOs (Unspent Transaction Outputs) from the block data.

2. Transaction Classification

The IndexingTask classifies deserialized transactions into types, converts them to typed objects, and prepares them for execution. OP_NET supports the following transaction types:

- **InteractionTransaction**: represents OP_NET transactions that invoke functions on existing smart contracts, processed via Block.executeTransaction() in the VMManager during the executeTransactions() loop. These include encoded calldata targeting deployed contract addresses, producing ContractEvaluation results with gas usage, state changes, and event logs committed to the StateMerkleTree and ReceiptMerkleTree.

- **DeploymentTransaction**: handles smart contract deployments on OP_NET via Block.deployContract(), extracting bytecode from transaction input for VMManager execution during the executeTransactions() pipeline. The immutable contract address is computed deterministically as `hash256(deployer_address ++ salt ++ hash256(bytecode))`, leveraging Bitcoin's double-SHA256 for counterfactual predictability and Taproot script-path spending with tweaked sender public keys. This design prevents code upgrades at deployed addresses, ensuring permanence through the block processing flow.

3. Smart Contract Execution

OP_NET transactions flow through the Execution Layer for deterministic smart contract processing.
- VMManager orchestrates isolated execution environments.
- ContractEvaluator enforces gas metering, storage caching, and reentrancy guards during executeTransaction(), producing ContractEvaluation results.
- RustContract executes bytecode in a sandboxed WebAssembly runtime .

A multi-level caching strategy optimizes storage operations from per-call values to persistent MongoDB storage.

4. State Tracking and Consensus

Execution results are tracked via StateMerkleTree and ReceiptMerkleTree, creating verifiable state summaries. The Consensus Layer enforces protocol rules through:
- OPNetConsensus: defines gas limits, contract constraints, and validation parameters
- EpochManager: validates proof-of-work submissions
- ReorgWatchdog: detects and recovers from chain reorganizations

5. Persistent Storage

The Storage Layer commits all data through repository patterns:
- Block headers and transactions via BlockRepository and TransactionRepository
- Contract state changes via VMStorage
- UTXOs via UnspentTransactionRepository

All data persists in MongoDB through VMMongoStorage.

6. Network Propagation and Mempool Management

After successful processing, P2PManager broadcasts the block, execution results, and witness signatures to connected peers. Concurrently, MempoolManager validates pending transactions, detects conflicts, and removes spent UTXOs to maintain mempool consistency.

3. FINDINGS

3.1. Flawed Validation Logic When Adding MLDSA Info to Cache Queue [CRITICAL] FIXED

SCOPE

- Commit: `82c998c05a1de5d3604c435698e7ae10b7a64203`
- Files:
  - `src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts#L185-L249`
  - `src/src/vm/VMManager.ts#L694-L726`

DESCRIPTION

```
// SharedInteractionParameters.ts
public async assignMLDSAToLegacy(vmManager: VMManager): Promise<void> {
    // ...
    if (this.mldsaLinkRequest.verifyRequest) {
        // ...
    } else {
        // Public key is null, this is just a post-quantum safe way to bind the keypair. We do not
force users to reveal due to the amount of data
        // required to do this action on-chain.
        await vmManager.addMLDSAInfoToStore({
            hashedPublicKey: this.mldsaLinkRequest.hashedPublicKey,
            legacyPublicKey: originalKey,
            tweakedPublicKey: tweakedKey,
            publicKey: null,
            level: this.mldsaLinkRequest.level,
            insertedBlockHeight: this.blockHeight,
            exposedBlockHeight: null,
        });
    }
    // From this point, even if the transaction fail, we still record the key link. (as long the
provided signatures are valid.)
    // The next action is safe here since if the public key (legacy) is already assigned OR the mldsa
hashed value
    // is already assigned, then this will throw.
    // Regenerate the real from.
    this._from = new Address(this.mldsaLinkRequest.hashedPublicKey, originalKey);
    // --> @auditor: The system assigns the hashedPublicKey to the transaction's _from value
}
```

```
// VMManager.ts
public async addMLDSAInfoToStore(mldsaPublicKey: IMLDSAPublicKey): Promise<void> {
    if (mldsaPublicKey.insertedBlockHeight !== this.vmBitcoinBlock.height) {
        throw new Error('MLDSA public key block height mismatch.');
    }
    // Verify we don't have a pending write on the mldsa public key.
    const address = new Address(mldsaPublicKey.hashedPublicKey, mldsaPublicKey.legacyPublicKey);
    if (this.mldsaToStore.has(address)) {
        return;
    }
    // Verify we don't have a pending write on this tweaked key.
    const tweakedAddress = new Address(address.tweakedPublicKeyToBuffer());
    if (this.mldsaToStoreLegacy.has(tweakedAddress)) {
        return;
    }
    // Verify it does not exist in the database.
    const exists = await this.shouldInsertMLDSAKey(
        mldsaPublicKey.hashedPublicKey,
        mldsaPublicKey.legacyPublicKey,
        mldsaPublicKey.level,
    );
    if (!exists) {
```

```
        this.mldsaToStore.set(address, {
            exposePublicKey: false,
            data: mldsaPublicKey,
        });
        this.mldsaToStoreLegacy.set(tweakedAddress, address);
    }
}
```

The `addMLDSAInfoToStore` function verifies whether a specific `tweakedAddress` or `hashedPublicKey` is already queued for addition to the `mldsaToStore` and `mldsaToStoreLegacy` cache. These values will be added to storage at the end of the block. However, when the `tweakedAddress` or `hashedPublicKey` already exists in the cache, the function simply returns without throwing an error. Processing continues, and the value is assigned as the `from` address of the transaction.

If an attacker intentionally creates a transaction in the same block using the same `hashedPublicKey` , this creates an exploitation risk for the user.

RECOMMENDATION

Review and update the `addMLDSAInfoToStore` function logic

UPDATES

- ***Jan 21, 2026***: The issue was fixed on commit `04a6488607101708f60e77dbb7681bc6c10c2348`

3.2. Decryption failure returns input message [HIGH] FIXED

SCOPE

- **Commit**: 99b96f68e080a03b651313035c9dc8df07299ed5
- **Files**: src/src/poa/networking/encryptem/EncryptemServer.ts

DESCRIPTION

```
#decrypt(
    msg: Buffer,
    senderPublicKey: Buffer,
    receiverPrivateKey: Buffer,
    signature: Buffer,
    senderSigningPublicKey: Buffer,
    auth: Buffer,
): Buffer | null {
    ...
    const verified = this.#verifySignature(cipher, signature, senderSigningPublicKey);
    if (verified) {
        return decryptedMessage;
    } else {
        this.error(`This message is not correctly signed. Authentication failed.`);
        return null;
    }
}
```

```
public decrypt(msg: Uint8Array): Uint8Array {
    ...
    const decryptedBuffer = this.#decrypt(
        data,
        this.#clientPublicKey,
        this.#serverPrivateKey,
        signature,
        this.#clientSignaturePublicKey,
        auth,
    );
    if (decryptedBuffer !== null) {
        msg = decryptedBuffer;
    }
    return msg;
}
```

When decryption fails, the function returns the original unencrypted message instead of throwing an error or returning null. This could lead to security vulnerabilities where failed decryption attempts silently pass through unencrypted data.

RECOMMENDATION

A better approach would be to either throw an error or return null to ensure the calling code knows the decryption failed.

UPDATED

- ***27 Nov, 2025***: The issue was fixed on commit 3a3363de735f64d504f64bb3d1d59c299b27e683

3.3. TaskId mismatch when handling message [MEDIUM] FIXED

SCOPE

- **Commit**: 99b96f68e080a03b651313035c9dc8df07299ed5
- **Files**: src/src/threading/Threader.ts

DESCRIPTION

```
private async executeMessageOnThread(
    message: ThreadMessageBase<MessageType>,
    selectedThread: Worker,
): Promise<ThreadData> {
    return new Promise((resolve: (value: ThreadData | PromiseLike<ThreadData>) => void) => {
        const taskId: string = this.generateRndTaskId();
        const timeout = setTimeout(() => {
            this.warn(`[A] Thread task ${taskId} timed out.`);
            resolve({
                error: true,
            });
        }, 30000);
        const task: ThreadTaskCallback = {
            timeout: timeout,
            resolve: resolve,
        };
        if (!message.taskId) { // If message.taskId was set
            message.taskId = taskId;
        }
        this.tasks.set(taskId, task); // <-- then taskId != message.taskId
        if (this.subChannels[selectedThread.threadId]) {
            this.subChannels[selectedThread.threadId].port2.postMessage(message);
        } else {
            selectedThread.postMessage(message);
        }
    });
}
```

`executeMessageOnThread()` generates a new `taskId` to set a callback for a task. However, it does not update `message.taskId` if this field is already set. This mismatch causes the callback lookup to fail in `onThreadMessage()` when processing the message response.

```
private onThreadMessage(thread: Worker, m: ThreadMessageBase<MessageType>): void {
    if (m.type === MessageType.LINK_THREAD) {
        //console.log('[THREAD] Link thread message received: ', m);
        return;
    }
    if (m.taskId && !m.toServer) {
        // So this lookup will fail
        const task: ThreadTaskCallback | undefined = this.tasks.get(m.taskId);
        if (task) {
            clearTimeout(task.timeout);
            task.resolve(m.data);
            this.tasks.delete(m.taskId);
        }
    } else {
        void this.onGlobalMessage(m, thread);
    }
}
```

UPDATES

- ***Nov 27, 2025***: The issue was fixed on commit eab65b87120306eb247d5925e116f32e1e16b238

3.4. Incorrect worker recreation logic [MEDIUM] FIXED

SCOPE

- **Commit**: 99b96f68e080a03b651313035c9dc8df07299ed5
- **Files**: src/src/threading/Threader.ts

DESCRIPTION

```
public async createThreads(): Promise<void> {
    const createPromises: Promise<void>[] = [];
    for (let i = 0; i < this.maxInstance; i++) {
        createPromises.push(
            this.createThread(i).then((worker) => {
                // If worker was successfully created, store it at index i
                if (worker) {
                    this.threads[i] = worker;
                }
            }),
        );
    }
    await Promise.safeAll(createPromises);
}
// ...
thread.on('exit', (exitCode: number | Error) => {
    this.error(
        `Thread #${i} died. {Target: ${this.target} | ExitCode -> ${exitCode}}`,
    );
    fs.appendFileSync(
        'threader.log',
        `Thread #${i} died. {Target: ${this.target} | ExitCode -> ${exitCode}}\n`,
    );
    // Find the correct thread by its ID in the array
    const idx = this.threads.findIndex((w) => w.threadId === thread.threadId);
    if (idx > -1) {
        this.threads.splice(idx, 1); // remove just that one
    }
    // Then re-create a new thread instance
    setTimeout(async () => {
         this.warn(
             `!!!!!!!!!!!!!! --------- Restarting thread #${i}... --------- !!!!!!!!!!!!!!`,
         );
         await this.createThread(i);
     }, 1000);
});
```

During initialization, the system calls `createThread` and assigns the created thread to `this.threads` . However, when handling thread exits, the system removes the thread from `this.threads` but executes `createThread` without assigning the newly recreated thread back to the `this.threads` array.

RECOMMENDATION

The recreated thread should be assigned back to the `this.threads` array using the original index.

UPDATES

- On commit `6ea4943d426e1244eec2f182e5c1870ce7869805` :

```
// Handle the worker exiting
thread.on('exit', (exitCode: number | Error) => {
    this.error(
        `Thread #${i} died. {Target: ${this.target} | ExitCode -> ${exitCode}}`,
    );
    fs.appendFileSync(
        'threader.log',
        `Thread #${i} died. {Target: ${this.target} | ExitCode -> ${exitCode}}\n`,
    );
```

```
    // Find the correct thread by its ID in the array
    const idx = this.threads.findIndex((w) => w.threadId === thread.threadId);
    if (idx > -1) {
        this.threads.splice(idx, 1); // remove just that one
    }

    // Then re-create a new thread instance
    setTimeout(async () => {
        this.warn(
            `!!!!!!!!!!!!!! --------- Restarting thread #${i}... --------- !!!!!!!!!!!!!!`,
        );
        await this.createThread(i);
     }, 1000);
});
```

With that updated code, when a thread exits, the system removes it from `this.threads` at its current index without preserving the slot. Recreating the thread later at that same index causes an incorrect state in `this.threads` .

- On commit `be65aa369602e57151c3c25d024aeff98ca084b5` : The issue was fixed by pushing to the end of the `this.threads` array after recreating it.

3.5. Incorrect logic in `getNextVMManager` [MEDIUM] FIXED

SCOPE

- **Commit**: `99b96f68e080a03b651313035c9dc8df07299ed5`
- **Files**: `src/src/utils/RPCSubWorker.ts`

DESCRIPTION

```
protected async getNextVMManager(tries: number = 0): Promise<VMManager> {
    if (tries > 10) {
        throw new Error('Failed to get a VMManager');
    }
    return new Promise<VMManager>(async (resolve) => {
        const startNumber = this.currentVMManagerIndex;
        let vmManager: VMManager | undefined;
        do {
            vmManager = this.vmManagers[this.currentVMManagerIndex];
            this.currentVMManagerIndex = (this.currentVMManagerIndex + 1) % this.CONCURRENT_VMS;
            if (!vmManager.busy() && vmManager.initiated) {
                break;
            }
        } while (this.currentVMManagerIndex !== startNumber);
        if (vmManager) {
            resolve(vmManager); // <- The vmManager may be currently busy or not fully initiated
            return;
        }
        this.warn(
            `High load detected. Try to increase your RPC thread limit or do fewer requests.`,
        );
        await this.sleep(100);
        resolve(await this.getNextVMManager(tries + 1));
    });
}
```

The function `getNextVMManager` iterates through all VM managers to find an available one. When looping through `this.vmManagers` , the function looks for a VM manager that meets the condition `!vmManager.busy() && vmManager.initiated` . However, if no VM manager meets this condition, it returns the current VM manager without validation, which could be busy or not fully initialized.

UPDATES

- ***Nov 27, 2025***: The issue was fixed on commit `8b7dccbd47fce70ed6a851159e04100e8340f7fd`

3.6. Flawed Transaction Grouping Logic in Block Transaction Processing [MEDIUM] FIXED

SCOPE

- Commit: `99b96f68e080a03b651313035c9dc8df07299ed5`
- Files:
  - `src/src/blockchain-indexer/processor/transaction/transaction-sorter/TransactionSorter.ts`
  - `src/src/blockchain-indexer/processor/transaction/transaction-sorter/TransactionGroupBuilder.ts`

DESCRIPTION

```
// TransactionGroupBuilder.ts#L1-L46
import { OPNetTransactionTypes } from '../enums/OPNetTransactionTypes.js';
import { Transaction } from '../Transaction.js';

export class TransactionGroupBuilder {
   public buildGroups(
       transactions: Transaction<OPNetTransactionTypes>[],
   ): Transaction<OPNetTransactionTypes>[][] {
       const groups: Transaction<OPNetTransactionTypes>[][] = [];
       const visited = new Set<string>();

       transactions.forEach((tx) => {
           if (!visited.has(tx.transactionIdString)) {
               const group: Transaction<OPNetTransactionTypes>[] = [];
               this.collectGroup(tx, transactions, group, visited);
               groups.push(group);
           }
       });

       return groups;
   }

   private collectGroup(
       tx: Transaction<OPNetTransactionTypes>,
       allTransactions: Transaction<OPNetTransactionTypes>[],
       group: Transaction<OPNetTransactionTypes>[],
       visited: Set<string>,
   ): void {
       if (visited.has(tx.transactionIdString)) return;

       visited.add(tx.transactionIdString);
       group.push(tx);
       const dependencies = this.findDependencies(allTransactions, tx.transactionId);
       dependencies.forEach((dep) => this.collectGroup(dep, allTransactions, group, visited));
   }

   private findDependencies(
       transactions: Transaction<OPNetTransactionTypes>[],
       txid: Buffer,
   ): Transaction<OPNetTransactionTypes>[] {
       return transactions.filter((tx) =>
           tx.inputs.some((input) =>
               input.originalTransactionId ? input.originalTransactionId.equals(txid) : false,
           ),
       );
   }
}
```

The `TransactionSorter` sorts transactions by fee for processing. Its `sortTransactions` function groups related transactions to maintain proper execution order, preventing scenarios where UTXOs are spent before being created. However, a critical flaw exists in the `TransactionGroupBuilder` logic: when a transaction uses inputs from multiple transactions in the same block, it's erroneously assigned to just one parent group instead of all relevant groups, potentially causing execution order problems.

Example Consider a block contains three transactions: [`tx_1` , `tx_2` , `tx_3`]

- Transaction `tx_1` produces `UTXO_1`
- Transaction `tx_2` produces `UTXO_2`
- Transaction `tx_3` spends both `UTXO_1` and `UTXO_2` as inputs

In this scenario, `tx_3` will be grouped with either `tx_1` or `tx_2` , but not both. This results in:

- Group 1: [`tx_1` , `tx_3`]
- Group 2: [`tx_2`]

UPDATES

- ***Jan 7, 2026***: The issue was fixed on commit `a5099fd39debe5191abd13194e06b621b70d0a72`

3.7. Gas Limit Check Uses Stale `gasUsed` Value `[MEDIUM]` `FIXED`

SCOPE

- Commit: `99b96f68e080a03b651313035c9dc8df07299ed5`
- File: `src/src/vm/runtime/ContractEvaluator.ts`

DESCRIPTION

```
export class ContractEvaluator extends Logger {
    private async calculateGasCostStore(evaluation: ContractEvaluation): Promise<void> {
        if (!evaluation.modifiedStorage) {
            return;
        }
        // TODO: Optimize using getStorageMultiple.
        let totalGasCost: bigint = 0n;
        let totalGasSpecial: bigint = 0n;
        for (const [address, states] of evaluation.modifiedStorage.entries()) {
            let cost: bigint = 0n;
            const isSpecialContract: boolean = !!(
                !evaluation.externalCall &&
                evaluation.specialContract &&
                evaluation.specialContract.freeGas &&
                evaluation.specialContract.address.equals(address)
            );
            const gasUsed = isSpecialContract ? evaluation.specialGasUsed : evaluation.gasUsed;
            for (const [key, value] of states) {
                let shouldThrow: boolean = false;
                if (
                    !isSpecialContract &&
                    evaluation.maxGas < gasUsed + cost + UPDATED_STORAGE_SLOT_GAS_COST
                ) {
                    shouldThrow = true;
                } else {
                    const currentValue = await this.getStorageState(evaluation, key, false);
                    if (currentValue === null) {
                        cost += NEW_STORAGE_SLOT_GAS_COST;
                    } else if (currentValue !== value) {
                        cost += UPDATED_STORAGE_SLOT_GAS_COST;
                    }
                    // Check if the gas used is less than the maximum.
                    if (!isSpecialContract && evaluation.maxGas < gasUsed + cost) {
                        shouldThrow = true;
                    }
                }
                if (shouldThrow) {
                    // Set the gas used to the maximum.
                    evaluation.setGasUsed(evaluation.paidMaximum);
                    throw new Error(`1. out of gas (consumed: ${evaluation.paidMaximum})`);
                }
            }
            if (isSpecialContract) {
                totalGasSpecial += cost;
            } else {
                totalGasCost += cost;
            }
        }
        evaluation.setFinalGasUsed(
            evaluation.gasUsed + totalGasCost,
            evaluation.specialGasUsed + totalGasSpecial,
        );
    }
}
```

`calculateGasCostStore()` iterates over `states` to accumulate storage costs and should stop once the consumed gas exceeds `evaluation.maxGas` . However, the check uses `evaluation.maxGas < gasUsed + cost` where `gasUsed` is read from `evaluation` (only updated at the end of the function). Because `gasUsed` is not incremented inside the loop, the condition can evaluate to true while the actual running gas has

already surpassed `evaluation.maxGas` , so the loop may continue past the limit. As a result, transactions that exceed the gas limit are permitted to finish, rather than being reverted as expected.

RECOMMENDATION

Update `gasUsed` within each iteration of the loop.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `d814358e8fb905e6c346879caacfbdda77bc12fa`

3.8. Lookup Failure When Tweaked Public Key Is Used Instead of Original [MEDIUM] FIXED

SCOPE

- Commit: `82c998c05a1de5d3604c435698e7ae10b7a64203`
- File:
  - `src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts#L374-L385`
  - `src/src/vm/VMManager.ts#L669-L692`

DESCRIPTION

The `getMLDSAByLegacy` function expects a legacy address value. However, when called from `getMLDSAPublicKeyFromLegacyKey`, a tweaked public key is passed instead. This causes the system to fail when reassigning existing MLDSA information for an address that has already submitted a link MLDSA public key request.

```
// The source code queries MLDSA Public Key information from storage using the legacy public key.
// src/src/vm/storage/databases/VMMongoStorage.ts
public getMLDSAByLegacy(
    publicKey: Buffer | Binary,
    blockHeight: bigint,
): Promise<IMLDSAPublicKey | null> {
    if (!this.mldsaPublicKeysRepository) {
        throw new Error('MLDSA Public Key repository not initialized');
    }

    return this.mldsaPublicKeysRepository.getByLegacyPublicKey(publicKey, blockHeight);
}
// src/src/db/repositories/MLDSAPublicKeysRepository.ts
public async getByLegacyPublicKey(
    legacyPublicKey: Buffer | Binary | string,
    blockHeight: bigint,
    currentSession?: ClientSession,
): Promise<IMLDSAPublicKey | null> {
    const binKey: Binary = this.toBinary(legacyPublicKey);
    const criteria: Document = {
        // @auditor: The function queries MLDSA Public Key information by the legacy public key.
        legacyPublicKey: binKey,
        insertedBlockHeight: { $lte: Long.fromBigInt(blockHeight) },
    };
    const result = await this.queryOne(criteria, currentSession);
    if (result) {
        delete (result as Document)._id;
    }
    return result ? this.parseResult(result) : null;
}
```

```
// The source code uses the tweaked public key for querying instead of the legacy public key.
// src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts
private async regenerateProvenance(vmManager: VMManager): Promise<void> {
    const originalKey = this.from.tweakedPublicKeyToBuffer();
    // Get the key assigned to the legacy address.
    const keyData = await vmManager.getMLDSAPublicKeyFromLegacyKey(originalKey);
    if (!keyData) {
        throw new Error(`OP_NET: No ML-DSA public key linked to the legacy address.`);
    }
    // Regenerate the real from.
    this._from = new Address(keyData.hashedPublicKey, originalKey);
}

// src/src/vm/VMManager.ts
public async getMLDSAPublicKeyFromLegacyKey(
    tweakedPublicKey: Buffer,
): Promise<IMLDSAPublicKey | null> {
```

```
    // ...
    const publicKeyData = this.vmStorage.getMLDSAByLegacy(
        // @auditor: The tweaked public key is passed to the getMLDSAByLegacy function
        tweakedPublicKey,
        this.vmBitcoinBlock.height,
    );
    // ...
}
```

RECOMMENDATION

Review the full call chain and correct the key type usage.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `d814358e8fb905e6c346879caacfbdda77bc12fa`

3.9. Missing `vmEvaluator` Cleanup Call `[LOW]` `FIXED`

SCOPE

- **Commit**: `99b96f68e080a03b651313035c9dc8df07299ed5`
- **Files**: `src/src/vm/VMManager.ts`

DESCRIPTION

```
public async clear(): Promise<void> {
    this.blockState = undefined;
    this.receiptState = undefined;

    this.pointerCache.clear();
    this.contractAddressCache.clear();
    this._blockHeaderValidator.clear();
    this.verifiedBlockHeights.clear();
    this.contractCache.clear();

    for (const vmEvaluator of this.vmEvaluators.values()) {
        await vmEvaluator; // <-- missing cleanup for vmEvaluator
    }

    this.vmEvaluators.clear();
}
```

In the above code, the cleanup for vmEvaluators was not handled properly. There should be something like `await vmEvaluator.cleanUp()`

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `d814358e8fb905e6c346879caacfbdda77bc12fa`

3.10. Incorrect Declaration of `subChannels` [LOW] FIXED

SCOPE

- **Commit**: `99b96f68e080a03b651313035c9dc8df07299ed5`
- **Files**: `src/src/threading/Threader.ts`

DESCRIPTION

```
private readonly subChannels: MessageChannel[] = [];
...
// keep track of subChannels by thread ID, or by index i
this.subChannels[thread.threadId] = messageChannel;
```

The `subChannels` array is being used as an object mapping, which mismatches its declared type.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `d814358e8fb905e6c346879caacfbdda77bc12fa`

3.11. Incorrect SSH Key Verification [LOW] FIXED

SCOPE

- **Commit**: `99b96f68e080a03b651313035c9dc8df07299ed5`
- **Files**: `src/src/ssh/client/SSHClient.ts`

DESCRIPTION

```
const allowedKey = this.allowedKeys.find((k) => k.type === algo);
if (!allowedKey) {
    this.rejectAuth(ctx);
    return;
}
if (!this.verifySafeBuffer(key.data, allowedKey.getPublicSSH())) {
    return this.rejectAuth(ctx);
}
```

If there are multiple allowed keys with the same type (algo), only the first key will be verified, causing other valid keys to be incorrectly marked as invalid.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `d814358e8fb905e6c346879caacfbdda77bc12fa`

3.12. `decrementPendingRequests()` may be called twice `[LOW]` `FIXED`

SCOPE

- **Commit**: `99b96f68e080a03b651313035c9dc8df07299ed5`
- **Files**:
  - `src/src/api/routes/api/v1/transaction/BroadcastTransaction.ts`
  - `src/src/api/routes/api/v1/states/Call.ts`

DESCRIPTION

Some places in the code have this pattern:

```
this.incrementPendingRequests();
try {
    // do some actions

    this.decrementPendingRequests(); // <-- 1. decrease counter

    // do another actions that could throw an error

    return { ... }
} catch (e) {
    this.decrementPendingRequests(); // <-- 2. decrease counter again!!!
    ...
}
```

In this case, the counter may be decremented twice if an error occurs after the first decrement. This could lead to incorrect tracking of pending requests since the counter could go negative.

RECOMMENDATION

A better approach would be to only decrement the counter once in the finally block to ensure it happens exactly one time regardless of success or failure.

```
this.incrementPendingRequests();
try {
  // do actions

  return { ... }
} catch (e) {
    ...
} finally {
  this.decrementPendingRequests();
}
```

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `e46f1fac3b6e402e42cc34fbbd2ca1d2011975d7`

3.13. Race Condition Can Prevent Cache Purge [LOW] FIXED

SCOPE

- Commit: `99b96f68e080a03b651313035c9dc8df07299ed5`
- File: `src/src/api/routes/api/v1/block/BlockRoute.ts`

DESCRIPTION

```
export abstract class BlockRoute {
    protected setToCache(
        height: SafeBigInt | string,
        data: Promise<BlockHeaderAPIDocumentWithTransactions>,
    ) {
        if (this.cachedBlocks.size() === 100) {
            this.purgeCache();
        }
        this.cachedBlocks.set(height, data);
    }
}
```

The `setToCache()` method uses the condition `this.cachedBlocks.size() === 100` to decide when to purge the cache. However, without any synchronization to make this operation atomic, a race condition can allow the cache size to exceed `100` . In such cases, the purge logic is never triggered.

RECOMMENDATION

Change the condition to `this.cachedBlocks.size() ⩾ 100` .

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `e46f1fac3b6e402e42cc34fbbd2ca1d2011975d7`

3.14. Incorrect Memory Allocation for `winner.solutionHash` [LOW] FIXED

SCOPE

- Commit: `8e204bdd66be138683d022f81c314239ea4b0ff2`
- File: `src/src/blockchain-indexer/processor/block/merkle/EpochMerkleTree.ts`

DESCRIPTION

```
public static epochDataToBytes(epochData: EpochData): Uint8Array {
    const baseSize = 64 + 8 + 8 + 8 + 32 + 32 + 8 + 32;
    const winnerSize = epochData.winner
        ? 32 + 2 + 32 + 32 + OPNetConsensus.consensus.EPOCH.GRAFFITI_LENGTH
        : 0;

    const writer = new BinaryWriter(baseSize + winnerSize);

    // ...

    writer.writeAddress(epochData.winner.publicKey); // 32
    writer.writeU16(epochData.winner.matchingBits & 0xffff); // 2
    writer.writeBytes(epochData.winner.salt); // 32
    writer.writeBytes(epochData.winner.solutionHash); // 32
    writer.writeBytes(resizedGraffiti); // OPNetConsensus.consensus.EPOCH.GRAFFITI_LENGTH
}
```

```
// In src/src/blockchain-indexer/processor/epoch/EpochManager.ts
private async finalizeEpoch() {
    const solutionHash = SHA1.hashBuffer(solution);

    // Create winner object
    const epochWinner: EpochSubmissionWinner = {
        epochNumber: epochNumber,
        publicKey: publicKey,
        solutionHash: solutionHash,
        salt: salt,
        matchingBits: matchingBits,
        graffiti: graffiti,
    };

    epoch.setWinner(epochWinner);
}
```

In `EpochMerkleTree.ts` , the `epochDataToBytes()` function currently allocates 32 bytes for `epochData.winner.solutionHash` . However, this is incorrect because `winner.solutionHash` is the output of `SHA1.hashBuffer()` , which produces a 20-byte digest. As a result, the allocated buffer is larger than necessary, leading to a mismatch between the expected and actual hash size during serialization.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `77dfb7a88eaddf3fa5d10fdaa9145280193804f7`

3.15. Problems handling `attestations` in `generateTree()` [LOW] FIXED

SCOPE

- Commit: `8e204bdd66be138683d022f81c314239ea4b0ff2`
- File: `src/src/blockchain-indexer/processor/block/merkle/EpochMerkleTree.ts`

DESCRIPTION

```
public generateTree(): void {
    // Sort attestations by timestamp (descending)
    this.attestations.sort((a, b) => b.timestamp - a.timestamp);

    // If we have no attestations, add dummy values
    if (this.attestations.length < 2) {
        this.addDummyAttestations();
    }
}

private addDummyAttestations(): void {
    // Add minimum attestations for tree generation
    const dummyAttestation1: Attestation = {
        type: AttestationType.EMPTY_ATTESTATION,
        blockNumber: this.epochData.startBlock,
        checksumRoot: Buffer.from(ZERO_HASH, 'hex'),
        signature: Buffer.alloc(64, 0),
        timestamp: 0,
        publicKey: Address.dead(),
    };

    const dummyAttestation2: Attestation = {
        type: AttestationType.EMPTY_ATTESTATION,
        blockNumber: this.epochData.endBlock - 1n,
        checksumRoot: Buffer.from(ZERO_HASH, 'hex'),
        signature: Buffer.alloc(64, 1),
        timestamp: 1,
        publicKey: Address.dead(),
    };

    this.attestations.push(dummyAttestation1, dummyAttestation2);
}
```

There are several issues with how the `attestations` array is handled:

1. The array is sorted before dummy attestations are inserted, but `addDummyAttestations()` can append new items afterward, potentially breaking the sorted order.
2. The comment suggests that dummy attestations are only added when there are no attestations, yet the actual condition is `this.attestations.length < 2` , which also inserts dummies when exactly one attestation is present.
3. `addDummyAttestations()` inserts two dummy attestations with increasing timestamps, which further disrupts the original sorting performed earlier.

UPDATES

- **Dec 22, 2025**: In commit 77dfb7a88eaddf3fa5d10fdaa9145280193804f7, the `sort()` operation was moved to occur after insertion, ensuring that the original order of attestations is preserved. Note that `addDummyAttestations()` still inserts elements in increasing order. Additionally, the comment was updated to clarify that dummy attestations are inserted when fewer than two are present (current behavior).

3.16. Inconsistent Logic Between `fromLevel` and `toLevel` Functions `[LOW]` `FIXED`

SCOPE

- `src/src/vm/mldsa/MLDSAMetadata.ts`

DESCRIPTION

```
/**
* Creates metadata from security level
*/
public static fromLevel(level: MLDSASecurityLevel): MLDSAPublicKeyMetadata {
    switch (level) {
        case MLDSASecurityLevel.LEVEL2:
            return MLDSAPublicKeyMetadata.MLDSA44;
        case MLDSASecurityLevel.LEVEL3:
            return MLDSAPublicKeyMetadata.MLDSA65;
        case MLDSASecurityLevel.LEVEL5:
            return MLDSAPublicKeyMetadata.MLDSA87;
        default:
            throw new Error('Invalid ML-DSA security level');
    }
}
/**
* Converts metadata to security level
*/
public static toLevel(metadata: MLDSAPublicKeyMetadata): number {
    switch (metadata) {
        case MLDSAPublicKeyMetadata.MLDSA44:
            return 0;
        case MLDSAPublicKeyMetadata.MLDSA65:
            return 1;
        case MLDSAPublicKeyMetadata.MLDSA87:
            return 2;
        default:
            return 0;
    }
}
```

The `fromLevel` and `toLevel` functions convert between metadata and security level values. However, their logic is inconsistent. In `toLevel`, the function should return `MLDSASecurityLevel.LEVEL2`, `MLDSASecurityLevel.LEVEL3`, and `MLDSASecurityLevel.LEVEL5` instead of `0`, `1`, and `2`.

RECOMMENDATION

Review and update the logic of the `fromLevel` and `toLevel` functions.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `77dfb7a88eaddf3fa5d10fdaa9145280193804f7`

3.17. Flawed Caching Logic in the `MLDSAPublicKeyCache` Class `[LOW]` `FIXED`

SCOPE

- `src/src/vm/cache/MLDSAPublicKeyCache.ts`

DESCRIPTION

```
// src/src/vm/cache/MLDSAPublicKeyCache.ts
private cache: AddressMap<Uint8Array> = new AddressMap<Uint8Array>();
private insertionOrder: Address[] = [];

public set(address: Address, publicKey: Uint8Array): void {
    const exists = this.cache.has(address);

    if (!exists && this.cache.size >= MLDSAPublicKeyCache.MAX_ITEMS) {
        this.evict();
    }

    this.cache.set(address, publicKey);

    if (exists) {
        const index = this.insertionOrder.indexOf(address);
    }
}

private evict(): void {
    const toRemove = this.insertionOrder.splice(
        0,
        MLDSAPublicKeyCache.MAX_ITEMS - MLDSAPublicKeyCache.EVICTION_THRESHOLD,
    );
    for (const addr of toRemove) {
        this.cache.delete(addr);
    }
}
```

The `set` function fails to push the `address` value to the `insertionOrder` array. As a result, the `insertionOrder` array remains empty. The cache size continues to grow indefinitely, and the `evict` action has no effect.

RECOMMENDATION

Include the logic that insert the `address` value to the `insertionOrder` array.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `77dfb7a88eaddf3fa5d10fdaa9145280193804f7`

3.18. Off-by-One Error in Remaining Byte Calculation [LOW] FIXED

SCOPE

- Commit: `82c998c05a1de5d3604c435698e7ae10b7a64203`
- File: `src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts`

DESCRIPTION

```
// src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts#L463-L466
        const binaryReader = new BinaryReader(data);
        const mldsaPublicKey = binaryReader.readBytes(32);
        const salt = binaryReader.readBytes(32);
        const bytesLeft = data.length - 65;
```

The code incorrectly subtracts 65 bytes from `data.length` when calculating `bytesLeft` , even though only 64 bytes are actually read from the input (32 bytes for `mldsaPublicKey` and 32 bytes for salt). This results in an off-by-one error that underestimates the number of remaining bytes by one.

RECOMMENDATION

Use `binaryReader.bytesLeft()` .

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `77dfb7a88eaddf3fa5d10fdaa9145280193804f7`

3.19. `exists()` length check breaks for hex string keys [LOW] FIXED

SCOPE

- Commit: `82c998c05a1de5d3604c435698e7ae10b7a64203`
- File: `src/src/db/repositories/MLDSAPublicKeysRepository.ts`

DESCRIPTION

```
public async exists(
    hashedPublicKey: Buffer | Binary | string,
    legacyPublicKey: Buffer | Binary | string,
): Promise<MLDSAPublicKeyExists> {
    if (hashedPublicKey.length !== 32 || legacyPublicKey.length !== 33) {
        throw new Error('Invalid public key lengths provided to compare existence');
    }
    const binHashed: Binary = this.toBinary(hashedPublicKey);
    const binLegacy: Binary = this.toBinary(legacyPublicKey);
    // ...
}
```

The `exists()` function uses the `length` property to validate the format of public keys; however, this approach is incorrect when the keys are provided as hex strings, since their length is doubled.

RECOMMENDATION

Update the validation logic to correctly handle multiple input types.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit `77dfb7a88eaddf3fa5d10fdaa9145280193804f7`

3.20. MLDSA Link Request Not Stored When Block Has No Successful Transactions [LOW] FIXED

SCOPE

- Commit: `82c998c05a1de5d3604c435698e7ae10b7a64203`
- Files:
  - `src/src/vm/VMManager.ts#L1166-L1216`
  - `src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts#L185-L249`

DESCRIPTION

```
// SharedInteractionParameters.ts
public async assignMLDSAToLegacy(vmManager: VMManager): Promise<void> {
    if (this.mldsaLinkRequest.verifyRequest) {
        // Reveal the public key assigned to the hashed version. Once revealed, the user may now use
mldsa signatures
        // in contracts and to authorize their opnet transaction.
        await this.verifyMLDSASignature(
            this.mldsaLinkRequest,
            originalKey,
            tweakedKey,
            chainId,
            vmManager,
        );
    } else {
        // Public key is null, this is just a post-quantum safe way to bind the keypair. We do not
force users to reveal due to the amount of data
        // required to do this action on-chain.
        await vmManager.addMLDSAInfoToStore({
            hashedPublicKey: this.mldsaLinkRequest.hashedPublicKey,
            legacyPublicKey: originalKey,
            tweakedPublicKey: tweakedKey,
            publicKey: null,
            level: this.mldsaLinkRequest.level,
            insertedBlockHeight: this.blockHeight,
            exposedBlockHeight: null,
        });
    }
    // From this point, even if the transaction fail, we still record the key link. (as long the
provided signatures are valid.)
    // The next action is safe here since if the public key (legacy) is already assigned OR the mldsa
hashed value
    // is already assigned, then this will throw.
    // Regenerate the real from.
    this._from = new Address(this.mldsaLinkRequest.hashedPublicKey, originalKey);
}
```

```
// VMManager.ts
private async saveBlockStateChanges(): Promise<void> {
    // ...

    /** Nothing to save. */
    if (!stateChanges) return;
    const storageToUpdate: AddressMap<Map<StoragePointer, [MemoryValue, string[]]>> =
        new AddressMap();
    // ...
    if (this.mldsaToStore.size) {
        const mldsaToStoreCopy = Array.from(this.mldsaToStore.values());
        // Everything from this point is considered safe. The keys have been verified before
insertion/update.
        await this.vmStorage.saveMLDSAPublicKeys(mldsaToStoreCopy);
    }
}
```

According to the comment in `assignMLDSAToLegacy` , the system should record the key link even if the transaction fails—as long as the signatures are valid. However, the `saveBlockStateChanges` function returns early when there are no state changes, preventing MLDSA info from being saved to storage. This happens when a block contains no successful transactions and all transactions are reverted during execution.

RECOMMENDATION

Separate the logic for persisting MLDSA link info from updating block state at the end of block processing.

UPDATES

- ***Jan 23, 2026***: The issue was fixed on commit `d082251dda4f69f8a00628f364dc842acd0607ee`

3.21. Redundant Legacy Signature Verification in Link Request Process [LOW] ACKED

SCOPE

- Commit: `82c998c05a1de5d3604c435698e7ae10b7a64203`
- File: `src/src/blockchain-indexer/processor/transaction/transactions/SharedInteractionParameters.ts#L185-L240`

DESCRIPTION

```
public async assignMLDSAToLegacy(vmManager: VMManager): Promise<void> {
    // ...
    const tweakedKey = this.from.tweakedPublicKeyToBuffer();
    const originalKey = this.from.originalPublicKeyBuffer();
    const chainId = getChainId(NetworkConverter.networkToBitcoinNetwork(this.network));

    const writer = new BinaryWriter();
    writer.writeU8(this.mldsaLinkRequest.level);
    writer.writeBytes(this.mldsaLinkRequest.hashedPublicKey);
    writer.writeBytes(tweakedKey);
    writer.writeBytes(originalKey);
    writer.writeBytes(OPNetConsensus.consensus.PROTOCOL_ID);
    writer.writeBytes(chainId);

    const message = writer.getBuffer();

    // First we check the schnorr signature
    const isValidSchnorr = MessageSigner.verifySignature(
        tweakedKey,
        message,
        this.mldsaLinkRequest.legacySignature,
    );

    if (!isValidSchnorr) {
        throw new Error(`OP_NET: Invalid ML-DSA legacy signature for public key link request.`);
    }

    // ...
}
```

During MLDSA link request processing, the system verifies that `legacySignature` is a valid signature signed by `tweakedKey` for a specific message format. This verification aims to prove the caller controls `tweakedKey` . However, since the transaction is already valid on the Bitcoin network, this verification is unnecessary and provides no additional value.

RECOMMENDATION

Review the `legacySignature` verification process and remove it if redundant.

3.22. Invalid Length Check for `targetHash` Parameter in `SubmitEpochRoute` [LOW] FIXED

SCOPE

- Commit: `99b96f68e080a03b651313035c9dc8df07299ed5`
- File: `src/src/api/routes/api/v1/epochs/SubmitEpochRoute.ts`

DESCRIPTION

```
// SubmitEpochRoute.ts
private validateHexStringLengths(params: SubmitEpochParamsAsObject): void {
    // ...

    // Target hash validation: must be 20 bytes (40 hex characters)
    if (!params.targetHash || typeof params.targetHash !== 'string') {
        throw new Error('Target hash must be a hex string');
    }
    const targetHashHex = params.targetHash.startsWith('0x')
        ? params.targetHash.slice(2)
        : params.targetHash;
    // @auditor: should check length against 40
    if (targetHashHex.length !== 64) {
        throw new Error(
            `Target hash must be 32 bytes (64 hex characters). Received ${targetHashHex.length}
characters`,
        );
    }
    // ...
}
```

The `targetHash` is the result of a SHA1 hash, which produces 20 bytes (40 hex characters). However, the validation logic incorrectly checks for 64 characters instead of 40.

RECOMMENDATION

Update the validation to check for a hex length of 40.

UPDATES

- ***Jan 22, 2026***: The issue was fixed on commit `59788e040ea6c19cff3ac5fc02e1eea7c7bc344a`

3.23. Missing `data.txid` produces empty buffer instead of 32-byte buffer [LOW] FIXED

SCOPE

- Commit: `d8b4c8231861085346db89a9ffdfade2dfa402cb`
- File: `src/src/blockchain-indexer/processor/transaction/inputs/TransactionInput.ts`

DESCRIPTION

```
export class TransactionInput implements ITransactionInput {
    constructor(data: VIn) {
        this.originalTransactionId = Buffer.from(data.txid || '', 'hex') || Buffer.alloc(32);
    }
}
```

When `data.txid` is missing, the code uses an empty string to create an empty buffer instead of falling back to allocating a 32-byte buffer (which is therefore never reached). This can lead to incorrect behavior if other parts of the system rely on the buffer length, for example when sorting transactions.

```
export class TransactionSorter {
    public sortTransactions(
        transactions: Transaction<OPNetTransactionTypes>[],
    ): Transaction<OPNetTransactionTypes>[] {
        // Filter block rewards and non-block rewards
        const blockRewards = transactions.filter((t) =>
            t.inputs.some((input) => input.originalTransactionId.length === 0),
        );
        const nonBlockRewards = transactions.filter((t) =>
            t.inputs.every((input) => input.originalTransactionId.length !== 0),
        );
    }
}
```

UPDATES

- ***Jan 23, 2026***: The issue was fixed on commit `2c04d290d03f642c852e43f03de239a040a896a4`

3.24. Inconsistent handling of the `address` and `addresses` parameters in the `GetLatestPendingTransactions` handler `[LOW]` `FIXED`

SCOPE

- Commit: `a67acb7b0e0ee51f3bb93e4766ed23631ae9efd7`
- Files: `src/src/api/routes/api/v1/mempool/GetLatestPendingTransactions.ts`

DESCRIPTION

The `GetLatestPendingTransactions` route handler behaves inconsistently when filtering by address. The `address` (singular) parameter triggers a `resolveAddress` feature that finds all derived wallet types for a given key, while the `addresses` (plural) parameter acts as a literal string filter.

```
// GetLatestPendingTransactions.ts
public async getData(
    params?: GetLatestPendingTransactionsParams,
): Promise<GetLatestPendingTransactionsResult> {
    // ...

    // If a single address is provided, auto-resolve all address types
    if (decoded.address && !addresses) {
        addresses = await this.resolveAddresses(decoded.address);
    }

    // ...
}
```

The current implementation creates a functional gap where the `resolveAddresses` feature is restricted to a single input. This fragmentation prevents users from monitoring multiple accounts simultaneously while still benefiting from automatic derived-address resolution, often necessitating manual lookups or redundant API calls.

RECOMMENDATION

Enhancing `resolveAddresses` to support bulk processing of address arrays, enabling the API to apply the same resolution logic to both singular and plural parameters.

UPDATES

- ***Mar 18, 2026***: The issue was fixed on commit `24f9456d790f540dbcaa3ac9a1ddff4cbd70c167`

3.25. Empty Array in `addresses` Parameter Suppresses Singular `address` Resolution `[LOW]` `FIXED`

SCOPE

- Commit: `a67acb7b0e0ee51f3bb93e4766ed23631ae9efd7`
- Files: `src/src/api/routes/api/v1/mempool/GetLatestPendingTransactions.ts`

DESCRIPTION

In the `GetLatestPendingTransactions` route, the logic that resolves derived addresses for a singular `address` parameter is bypassed when the plural `addresses` parameter is provided as an empty array `[]`. Because the check `!addresses` returns `false` for an empty array, the API ignores the `address` parameter and queries storage with an empty filter.

```
// GetLatestPendingTransactions.ts
public async getData(
    params?: GetLatestPendingTransactionsParams,
): Promise<GetLatestPendingTransactionsResult> {
    // ...
    // If a single address is provided, auto-resolve all address types
    if (decoded.address && !addresses) {
        addresses = await this.resolveAddresses(decoded.address);
    }
    if (addresses && addresses.length > Config.API.MEMPOOL.MAX_ADDRESSES) {
        throw new Error(`Too many addresses. Maximum is ${Config.API.MEMPOOL.MAX_ADDRESSES}.`);
    }
    const txs = await this.storage.getLatestPendingTransactions(addresses, limit);
    return {
        transactions: MempoolTransactionConverter.convertMany(txs),
    };
}
```

RECOMMENDATION

The logic should be updated to strictly check for a non-empty list before bypassing the singular `address` resolution. This ensures that an empty plural parameter does not accidentally suppress the intended resolution of a primary address.

UPDATES

- ***Mar 18, 2026***: The issue was fixed on commit `24f9456d790f540dbcaa3ac9a1ddff4cbd70c167`

3.26. Use Loop Instead of Recursion for Retry Logic [INFO] FIXED

SCOPE

- **Commit**: 99b96f68e080a03b651313035c9dc8df07299ed5

DESCRIPTION

Several places in the code that use recursion for retry logic can lead to a stack overflow with many retries. For example:

```
private async getBlockTransactionDataAndRetryIfNull(
    blockHash: string,
    retries: number = 0,
): Promise<BlockDataWithTransactionData | null> {
    try {
        const resp = await this.rpc.getBlockInfoWithTransactionData(blockHash);
        if (resp == null) {
            throw new Error(`Error fetching block ${blockHash}. (response is null)`);
        }

        return resp;
    } catch {
        if (retries >= this.maxRetries) {
            throw new Error(`Error fetching block ${blockHash}. (response is null)`);
        }

        return this.getBlockTransactionDataAndRetryIfNull(blockHash, retries + 1);
    }
}
```

A better approach would be to use a while loop, which maintains a constant stack depth regardless of the retry count.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit 77dfb7a88eaddf3fa5d10fdaa9145280193804f7

3.27. Unsafe algorithms in SSH config [INFO] FIXED

SCOPE

- **Commit**: 99b96f68e080a03b651313035c9dc8df07299ed5
- **Files**: src/src/ssh/SSH.ts

DESCRIPTION

```
private ssh2Configs(): ServerConfig {
    ...
    return {
        ...
        algorithms: {
            kex: [...],
            cipher: [...],
            serverHostKey: [
                'ssh-ed25519',
                'ecdsa-sha2-nistp256',
                'ecdsa-sha2-nistp384',
                'ecdsa-sha2-nistp521',
                'rsa-sha2-512',
                'rsa-sha2-256',
                'ssh-rsa', // <-- this algorithm is unsafe and deprecated
                'ssh-rsa', // <-- duplicated
            ],
            hmac: [
                'hmac-sha2-256-etm@openssh.com',
                'hmac-sha2-512-etm@openssh.com',
                'hmac-sha1-etm@openssh.com', // <-- considered unsecure
                'hmac-sha2-256',
                'hmac-sha2-512',
                'hmac-sha1', // <-- considered unsecure
            ],
            compress: ['zlib', 'zlib@openssh.com', 'none'],
        },
        debug: () => {},
    };
}
```

Some algorithms in SSH config is unsecured and must be removed. (See this changelog https://www.openssh.com/txt/release-8.2)

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit 77dfb7a88eaddf3fa5d10fdaa9145280193804f7

3.28. Vulnerabilities in Processing Transaction Input/Output Amounts Exceeding Limits [INFO] FIXED

SCOPE

- Commit: 99b96f68e080a03b651313035c9dc8df07299ed5
- Files:
  - src/src/vm/VMManager.ts#L403-L404
  - src/src/vm/VMManager.ts#L509-L510
  - src/src/blockchain-indexer/processor/transaction/Transaction.ts#L114-L124

DESCRIPTION

```
// Pass transaction parameters to the vmEvaluator
inputs: contractTransaction.strippedInputs,
outputs: contractTransaction.strippedOutputs,

public get strippedInputs(): StrippedTransactionInput[] {
    return this.inputs
        .slice(0, OPNetConsensus.consensus.VM.UTXOS.MAXIMUM_INPUTS)
        .map((input) => input.toStripped());
}

public get strippedOutputs(): StrippedTransactionOutput[] {
     const outputs = this.outputs
        .slice(0, OPNetConsensus.consensus.VM.UTXOS.MAXIMUM_OUTPUTS)
        .map((output) => output.toStripped());

    return outputs.filter((output): output is StrippedTransactionOutput => !!output);
}
```

When the OP_NET node fetches transactions from Bitcoin blockchains, it passes UTXO inputs and outputs to the `vmEvaluator` for execution. However, an issue exists: if the number of inputs/outputs exceeds the consensus-defined maximum, the system silently processes only the first `MAXIMUM_INPUTS` / `MAXIMUM_OUTPUTS` without generating any error or notification. This silent truncation can lead to unexpected transaction behavior.

UPDATES

- ***Dec 22, 2025***: The issue was fixed on commit 77dfb7a88eaddf3fa5d10fdaa9145280193804f7

3.29. Error message incorrectly references SHA-1 instead of SHA-256 [INFO] FIXED

SCOPE

- Commit: `d8b4c8231861085346db89a9ffdfade2dfa402cb`
- File: `src/src/api/routes/api/v1/epochs/EpochRoute.ts`

DESCRIPTION

```
const includeSubmissions = params.includeSubmissions ?? false;
if ('hash' in params) {
    if (typeof params.hash !== 'string') {
        throw new Error('Invalid hash parameter: expected a string');
    }
    const cleanHash: string | undefined = params.hash?.replace('0x', '');
    if (cleanHash.length !== 64) {
        throw new Error('Invalid hash length. Expected 64 hex characters for SHA-1 hash');
    }
    return { epochHash: cleanHash, includeSubmissions };
}
```

`cleanHash` is 64 characters long, which means it's a SHA-256 hash, so the error message should be updated from SHA-1 to SHA-256.

UPDATES

- ***Jan 23, 2026***: The issue was fixed on commit `34bb4c6c1192b0117c03d45fb0ac5bb2c599edf2`

4. VERSION HISTORY

Version	Date	Status/Change	Changed by
0.1	**Aug 1, 2025**	Draft Report	Verichains Lab
0.2	**Oct 27, 2025**	Draft Report	Verichains Lab
0.3	**Nov 21, 2025**	Draft Report	Verichains Lab
0.4	**Dec 29, 2025**	Draft Report	Verichains Lab
0.5	**Jan 8, 2026**	Draft Report	Verichains Lab
0.6	**Mar 16, 2026**	Draft Report	Verichains Lab
1.0	**Jan 19, 2026**	Private Report	Verichains Lab
1.1	**Mar 19, 2026**	Public Report	Verichains Lab